Identificação do projeto

Título do projeto: Vendas, promoções e sobra de produção em uma rede de padarias
Integrantes do grupo: Helena Picinin, Marco Capote, Raissa Queiroz e Rafael Sene
Turma: [a preencher]
Descrição breve do problema ou contexto: Padarias vendem produtos que estragam rápido e precisam decidir todos os dias quanto produzir. O que sobra vira prejuízo e o que falta vira venda perdida. Usamos dois anos de vendas de uma rede com três lojas para entender como horário, calendário, clima e promoções se relacionam com faturamento, margem e sobra de produtos.

RA1 — Data Ingestion, Cleaning and Initial Analysis

Nesta etapa, carregue, prepare e explore os dados do projeto.

1. Contexto e objetivo

Apresente brevemente o problema investigado e o que o grupo pretende descobrir com os dados.

Resposta do grupo:

Contexto. Boa parte do que uma padaria produz precisa ser vendida no mesmo dia ou em poucos dias. Pães, bolos e salgados perdem qualidade rápido, então errar a quantidade produzida custa caro: se sobra, o produto é descartado; se falta, o cliente vai embora sem comprar. Promoções ajudam a girar o estoque, mas diminuem a margem. E a procura muda com o horário, o dia da semana, a estação do ano, o clima e as datas comemorativas.

Problema. A rede que vamos analisar tem três lojas e registra, para cada item vendido, quanto foi produzido do lote e quanto sobrou. Queremos entender em quais momentos e condições a padaria vende mais, quais produtos concentram a sobra e se as promoções compensam a perda de margem. A ideia é que essas respostas ajudem a planejar a produção e as promoções.

Objetivo do RA1. Deixar pronta uma base confiável de vendas por item, com os problemas de qualidade identificados, tratados e justificados, para responder às perguntas abaixo nas próximas etapas.

Recorte.

- Período: 01/01/2023 a 31/12/2024.
- Unidade de análise: um produto vendido dentro de um pedido (cada linha da base).
- Abrangência: 3 lojas (`ST01` , `ST02` , `ST03`), 35 produtos de 7 categorias e 500 clientes.
- Limite importante: a base é sintética, gerada pelo autor para fins educacionais. Os resultados servem para exercitar o processo analítico e não descrevem uma padaria real. A moeda não é informada (pelo contexto, rupia indiana).

Perguntas analíticas.

ID	Pergunta a investigar	Dados necessários
P1	Como o faturamento e a quantidade de itens vendidos variam ao longo do dia, entre os dias da semana e entre as estações, em cada loja, em 2023 e 2024?	`Timestamp` , `Hour` , `Day_of_Week` , `Weekend` , `Season` , `Store_ID` , `Total_Bill` , `Quantity`
P2	Dias de festival e as condições do tempo (tipo de clima e temperatura) estão associados a mudanças no volume vendido e no faturamento de cada categoria?	`Festival` , `Weather` , `Temperature` , `Category` , `Quantity` , `Total_Bill`
P3	Os itens vendidos com promoção, e cada tipo de promoção, vendem quantidade suficiente para compensar a margem menor?	`Promotion_Applied` , `Promotion_Type` , `Discount_Percentage` , `Quantity` , `Profit`
P4	Quais produtos, categorias e níveis de perecibilidade concentram os maiores percentuais de sobra em relação ao que foi produzido?	`Product` , `Category` , `Perishability` , `Shelf_Life_Days` , `Units_Produced` , `Unsold_Units` , `Waste_Pct`

Tipo de análise. Descritiva (P1 e P4) e diagnóstica (P2 e P3).

Viabilidade. A base traz todos os campos que as quatro perguntas pedem, incluindo data e hora de cada venda, o que também permite simular a chegada de vendas no RA2. As lacunas que já conhecemos: o clima é registrado por dia e não por hora; os campos de produção e sobra se referem ao lote do produto e não ao pedido; e não sabemos a moeda nem onde ficam as lojas.

2. Fonte e ingestão dos dados

- Identifique a fonte dos dados, incluindo link ou referência de acesso.
- Descreva brevemente o dataset e o que seus registros representam.
- Explique como os dados foram carregados e implemente a ingestão.

Resposta do grupo:

Fonte. *Bakery Sales, Customer & Inventory Analytics Dataset*, publicado no Kaggle pelo usuário `sdcloud` : https://www.kaggle.com/datasets/sdcloud/historical-bakery-sales-dataset (acesso em 20/10/2026). Licença CC0 (domínio público), por isso a cópia do arquivo bruto pode ficar no repositório.

Sobre o dataset.

- Formato: CSV separado por vírgula, codificado em UTF-8 com BOM (`bakery_sales_dataset.csv` , cerca de 4,2 MB). A página também oferece uma cópia em Excel, que não usamos.
- Volume: 19.321 linhas e 40 colunas, de 01/01/2023 a 31/12/2024.

- O autor informa que os dados são sintéticos e foram criados para estudo de vendas, previsão de demanda, estoque e desperdício.
- Cada linha é **um produto dentro de um pedido**. Um pedido (`Order_ID`) com três produtos aparece em três linhas, e as informações do pedido (cliente, data, hora, loja, clima, forma de pagamento) se repetem em cada uma delas.

Ingestão. O notebook lê o arquivo de `data/raw/bakery_sales_dataset.csv` . Se ele não existir (por exemplo, ao rodar no Colab sem clonar o repositório), é baixado do endereço público de download do Kaggle, que não exige login. A leitura usa `encoding="utf-8-sig"` para descartar o BOM; sem isso, o nome da primeira coluna viria com um caractere invisível. O DataFrame `df_original` guarda os dados exatamente como chegaram e não é alterado ao longo do notebook.

In [1]:
```
import urllib.request
from pathlib import Path

import numpy as np
import pandas as pd
from IPython.display import display

pd.set_option("display.max_columns", None)
pd.set_option("display.width", 200)
```

In [2]:
```
URL_KAGGLE = (
    "https://www.kaggle.com/api/v1/datasets/download/"
    "sdcloud/historical-bakery-sales-dataset/bakery_sales_dataset.csv"
)
ARQUIVO_BRUTO = Path("data/raw/bakery_sales_dataset.csv")

if not ARQUIVO_BRUTO.exists():
    print("Arquivo local não encontrado. Baixando do Kaggle...")
    ARQUIVO_BRUTO.parent.mkdir(parents=True, exist_ok=True)
    urllib.request.urlretrieve(URL_KAGGLE, ARQUIVO_BRUTO)

df_original = pd.read_csv(ARQUIVO_BRUTO, sep=",", encoding="utf-8-sig")

print(f"Arquivo: {ARQUIVO_BRUTO} ({ARQUIVO_BRUTO.stat().st_size / 1_000_000:.1f} MB)")
print(f"Linhas: {df_original.shape[0]} | Colunas: {df_original.shape[1]}")

# com 40 colunas fica mais fácil ler as primeiras linhas na vertical
df_original.head(3).T
```

```
Arquivo: data\raw\bakery_sales_dataset.csv (4.2 MB)
Linhas: 19321 | Colunas: 40
```

Out[2]:

	0	1	2
Order_ID	ORD00001	ORD00001	ORD00002
Customer_ID	CUST0328	CUST0328	CUST0016
Customer_Age	46	46	21
Customer_Gender	Other	Other	Female
Customer_Segment	Young Professional	Young Professional	Student
Loyalty_Member	Yes	Yes	No
Date	2023-01-01	2023-01-01	2023-01-01
Year	2023	2023	2023
Month	1	1	1
Day	1	1	1
Day_of_Week	Sunday	Sunday	Sunday
Weekend	Yes	Yes	Yes
Hour	8	8	15
Time	8:47	8:47	15:14
Season	Winter	Winter	Winter
Festival	New Year	New Year	New Year
Weather	Foggy	Foggy	Foggy
Temperature	21.2	21.2	21.2
Product	Pastry	Biscuit	Chocolate Cake
Category	Pastry & Sweet	Indian Bakery	Cakes
Quantity	6	3	1
Unit_Price	62.49	30.14	425.49
Cost_Price	29.72	12.63	256.56
Discount_Percentage	14.7	13.9	15.3
Selling_Price	53.3	25.95	360.39
Total_Bill	319.8	77.85	360.39
Payment_Method	Cash	Cash	UPI
Promotion_Applied	Yes	Yes	Yes
Promotion_Type	Festival	Festival	Festival
Shelf_Life_Days	3	64	3
Storage_Type	Refrigerated	Dry Storage	Refrigerated
Perishability	Very High	Low	Very High
Store_ID	ST02	ST02	ST03
Units_Produced	8	4	3
Units_Sold	6	3	2
Unsold_Units	2	1	1
Stock_Available	2	1	1
Waste_Pct	25.0	25.0	33.3
Profit	141.48	39.96	103.83
Customer_Rating	4	4	4

Antes de procurar problemas, montamos um dicionário com o significado de cada coluna. Os significados vêm da descrição do dataset no Kaggle e de conferências feitas na própria base (por exemplo, `Total_Bill` é sempre `Selling_Price × Quantity`, como mostramos na seção 3).

In [3]:
```
dicionario = pd.DataFrame([
    ("Order_ID", "pedido", "Identificador do pedido (se repete para cada produto do pedido)"),
    ("Customer_ID", "cliente", "Identificador do cliente"),
    ("Customer_Age", "cliente", "Idade do cliente, em anos"),
    ("Customer_Gender", "cliente", "Gênero informado: Male, Female ou Other"),
    ("Customer_Segment", "cliente", "Perfil: Student, Young Professional, Family ou Senior
Citizen"),
    ("Loyalty_Member", "cliente", "Se participa do programa de fidelidade (Yes/No)"),
    ("Date", "calendário", "Data da venda (AAAA-MM-DD)"),
```

```
    ("Year / Month / Day", "calendário", "Ano, mês e dia extraídos da data"),
    ("Day_of_Week", "calendário", "Dia da semana, em inglês"),
    ("Weekend", "calendário", "Se a venda foi no sábado ou domingo (Yes/No)"),
    ("Hour", "calendário", "Hora da venda (0 a 23)"),
    ("Time", "calendário", "Horário da venda (H:MM)"),
    ("Season", "calendário", "Estação: Winter, Summer, Monsoon ou Autumn"),
    ("Festival", "calendário", "Festival ou feriado do dia; vazio quando é um dia comum"),
    ("Weather", "clima", "Condição do tempo no dia"),
    ("Temperature", "clima", "Temperatura do dia, em °C"),
    ("Product", "produto", "Nome do produto"),
    ("Category", "produto", "Categoria do produto"),
    ("Shelf_Life_Days", "produto", "Vida útil do lote, em dias"),
    ("Storage_Type", "produto", "Forma de armazenamento"),
    ("Perishability", "produto", "Nível de perecibilidade: Low, Medium, High ou Very High"),
    ("Quantity", "venda", "Unidades do produto no pedido"),
    ("Unit_Price", "venda", "Preço de tabela por unidade"),
    ("Cost_Price", "venda", "Custo por unidade"),
    ("Discount_Percentage", "venda", "Desconto aplicado ao item, em %"),
    ("Selling_Price", "venda", "Preço por unidade depois do desconto"),
    ("Total_Bill", "venda", "Valor do item no pedido (Selling_Price × Quantity)"),
    ("Profit", "venda", "Lucro do item ((Selling_Price − Cost_Price) × Quantity)"),
    ("Payment_Method", "venda", "Forma de pagamento: Cash, Card, UPI ou Wallet"),
    ("Promotion_Applied", "promoção", "Se havia promoção (Yes/No)"),
    ("Promotion_Type", "promoção", "Tipo da promoção; vazio quando não houve promoção"),
    ("Store_ID", "loja", "Loja onde a venda aconteceu"),
    ("Units_Produced", "lote", "Unidades produzidas no lote do produto"),
    ("Units_Sold", "lote", "Unidades vendidas desse lote"),
    ("Unsold_Units", "lote", "Unidades que sobraram (produzidas − vendidas)"),
    ("Stock_Available", "lote", "Estoque disponível registrado"),
    ("Waste_Pct", "lote", "Percentual de sobra (Unsold_Units ÷ Units_Produced × 100)"),
    ("Customer_Rating", "avaliação", "Nota dada pelo cliente, de 1 a 5"),
], columns=["coluna", "grupo", "significado"])

display(dicionario.style.hide(axis="index").set_properties(**{"text-align": "left"}))
```

coluna	grupo	significado
Order_ID	pedido	Identificador do pedido (se repete para cada produto do pedido)
Customer_ID	cliente	Identificador do cliente
Customer_Age	cliente	Idade do cliente, em anos
Customer_Gender	cliente	Gênero informado: Male, Female ou Other
Customer_Segment	cliente	Perfil: Student, Young Professional, Family ou Senior Citizen
Loyalty_Member	cliente	Se participa do programa de fidelidade (Yes/No)
Date	calendário	Data da venda (AAAA-MM-DD)
Year / Month / Day	calendário	Ano, mês e dia extraídos da data
Day_of_Week	calendário	Dia da semana, em inglês
Weekend	calendário	Se a venda foi no sábado ou domingo (Yes/No)
Hour	calendário	Hora da venda (0 a 23)
Time	calendário	Horário da venda (H:MM)
Season	calendário	Estação: Winter, Summer, Monsoon ou Autumn
Festival	calendário	Festival ou feriado do dia; vazio quando é um dia comum
Weather	clima	Condição do tempo no dia
Temperature	clima	Temperatura do dia, em °C
Product	produto	Nome do produto
Category	produto	Categoria do produto
Shelf_Life_Days	produto	Vida útil do lote, em dias
Storage_Type	produto	Forma de armazenamento
Perishability	produto	Nível de perecibilidade: Low, Medium, High ou Very High
Quantity	venda	Unidades do produto no pedido
Unit_Price	venda	Preço de tabela por unidade
Cost_Price	venda	Custo por unidade
Discount_Percentage	venda	Desconto aplicado ao item, em %
Selling_Price	venda	Preço por unidade depois do desconto
Total_Bill	venda	Valor do item no pedido (Selling_Price × Quantity)
Profit	venda	Lucro do item ((Selling_Price − Cost_Price) × Quantity)
Payment_Method	venda	Forma de pagamento: Cash, Card, UPI ou Wallet
Promotion_Applied	promoção	Se havia promoção (Yes/No)
Promotion_Type	promoção	Tipo da promoção; vazio quando não houve promoção
Store_ID	loja	Loja onde a venda aconteceu
Units_Produced	lote	Unidades produzidas no lote do produto
Units_Sold	lote	Unidades vendidas desse lote
Unsold_Units	lote	Unidades que sobraram (produzidas − vendidas)
Stock_Available	lote	Estoque disponível registrado
Waste_Pct	lote	Percentual de sobra (Unsold_Units ÷ Units_Produced × 100)
Customer_Rating	avaliação	Nota dada pelo cliente, de 1 a 5

3. Exploração inicial dos dados

Investigue a quantidade de registros e colunas, os tipos de dados, os valores ausentes e outras características relevantes da base.

Nesta seção apenas olhamos os dados, sem alterar nada. Começamos pelo tamanho da base, pelos tipos que o Pandas inferiu e pelos valores ausentes.

In [4]:
```
print(f"Linhas: {df_original.shape[0]} | Colunas: {df_original.shape[1]}")
print(f"Pedidos distintos: {df_original['Order_ID'].nunique()}")
print(f"Clientes distintos: {df_original['Customer_ID'].nunique()}")
print(f"Produtos distintos: {df_original['Product'].nunique()}")
print(f"Período: {df_original['Date'].min()} a {df_original['Date'].max()}")

visao_geral = pd.DataFrame({
```

```
    "tipo inferido": df_original.dtypes.astype(str),
    "ausentes": df_original.isna().sum(),
    "ausentes (%)": (df_original.isna().mean() * 100).round(1),
    "valores distintos": df_original.nunique(),
})
display(visao_geral)
```

```
Linhas: 19321 | Colunas: 40
Pedidos distintos: 6576
Clientes distintos: 500
Produtos distintos: 35
Período: 2023-01-01 a 2024-12-31
```

	tipo inferido	ausentes	ausentes (%)	valores distintos
Order_ID	object	0	0.0	6576
Customer_ID	object	0	0.0	500
Customer_Age	int64	0	0.0	57
Customer_Gender	object	0	0.0	3
Customer_Segment	object	0	0.0	4
Loyalty_Member	object	0	0.0	2
Date	object	0	0.0	731
Year	int64	0	0.0	2
Month	int64	0	0.0	12
Day	int64	0	0.0	31
Day_of_Week	object	0	0.0	7
Weekend	object	0	0.0	2
Hour	int64	0	0.0	15
Time	object	0	0.0	872
Season	object	0	0.0	4
Festival	object	16006	82.8	9
Weather	object	0	0.0	6
Temperature	float64	0	0.0	305
Product	object	0	0.0	35
Category	object	0	0.0	7
Quantity	int64	0	0.0	17
Unit_Price	float64	0	0.0	7711
Cost_Price	float64	0	0.0	6232
Discount_Percentage	float64	0	0.0	292
Selling_Price	float64	0	0.0	8657
Total_Bill	float64	0	0.0	11349
Payment_Method	object	0	0.0	4
Promotion_Applied	object	0	0.0	2
Promotion_Type	object	13826	71.6	5
Shelf_Life_Days	int64	0	0.0	498
Storage_Type	object	0	0.0	3
Perishability	object	0	0.0	4
Store_ID	object	0	0.0	3
Units_Produced	int64	0	0.0	45
Units_Sold	int64	0	0.0	30
Unsold_Units	int64	0	0.0	16
Stock_Available	int64	0	0.0	17
Waste_Pct	float64	0	0.0	100
Profit	float64	0	0.0	8460
Customer_Rating	int64	0	0.0	5

In [5]:
```
colunas_numericas = df_original.select_dtypes("number").columns
display(df_original[colunas_numericas].describe().T.round(2))
```

	count	mean	std	min	25%	50%	75%	max
Customer_Age	19321.0	35.54	13.73	16.00	25.00	34.00	45.00	73.00
Year	19321.0	2023.51	0.50	2023.00	2023.00	2024.00	2024.00	2024.00
Month	19321.0	6.56	3.48	1.00	4.00	7.00	10.00	12.00
Day	19321.0	15.87	8.77	1.00	8.00	16.00	24.00	31.00
Hour	19321.0	12.86	3.94	7.00	9.00	12.00	17.00	21.00
Temperature	19321.0	26.99	8.81	8.10	21.20	27.70	33.20	44.90
Quantity	19321.0	2.23	1.78	1.00	1.00	2.00	3.00	20.00
Unit_Price	19321.0	87.26	110.48	7.60	31.02	45.09	76.65	472.42
Cost_Price	19321.0	49.28	65.97	3.27	16.90	23.64	52.64	296.28
Discount_Percentage	19321.0	4.35	6.79	0.00	0.00	0.00	7.50	30.00
Selling_Price	19321.0	83.37	105.76	5.68	30.40	42.66	72.03	472.42
Total_Bill	19321.0	139.27	157.77	13.12	52.12	82.80	146.54	1885.96
Shelf_Life_Days	19321.0	61.12	103.61	1.00	3.00	5.00	82.00	729.00
Units_Produced	19321.0	4.22	3.64	2.00	2.00	3.00	5.00	49.00
Units_Sold	19321.0	3.05	3.04	1.00	1.00	2.00	4.00	37.00
Unsold_Units	19321.0	1.17	0.78	1.00	1.00	1.00	1.00	16.00
Stock_Available	19321.0	1.13	0.75	1.00	1.00	1.00	1.00	20.00
Waste_Pct	19321.0	34.12	12.73	4.20	25.00	33.30	50.00	50.00
Profit	19321.0	59.20	65.24	-0.55	20.69	36.76	65.92	767.60
Customer_Rating	19321.0	3.83	0.95	1.00	3.00	4.00	4.00	5.00

Duplicatas. O `Order_ID` se repete por natureza, porque cada produto do pedido ocupa uma linha. Então contar repetições de `Order_ID` não mede duplicidade. A chave que identifica um registro é o par `Order_ID` + `Product`. Verificamos as duas coisas, além de linhas totalmente iguais.

In [6]:
```
print("Linhas totalmente repetidas:", df_original.duplicated().sum())
print("Repetições de Order_ID:", df_original["Order_ID"].duplicated().sum())
print("Repetições de Order_ID + Product:", df_original.duplicated(subset=["Order_ID",
"Product"]).sum())

itens_por_pedido = df_original.groupby("Order_ID").size()
print("\nQuantidade de produtos por pedido:")
print(itens_por_pedido.value_counts().sort_index().to_string())

print("\nExemplo: o pedido ORD00002 tem três produtos diferentes")
display(df_original.loc[df_original["Order_ID"] == "ORD00002",
                        ["Order_ID", "Customer_ID", "Date", "Time", "Product", "Quantity",
"Total_Bill"]])
```

```
Linhas totalmente repetidas: 0
Repetições de Order_ID: 12745
Repetições de Order_ID + Product: 0

Quantidade de produtos por pedido:
1    1105
2    1753
3    1666
4    1045
5     641
6     235
7     131

Exemplo: o pedido ORD00002 tem três produtos diferentes
```

	Order_ID	Customer_ID	Date	Time	Product	Quantity	Total_Bill
2	ORD00002	CUST0016	2023-01-01	15:14	Chocolate Cake	1	360.39
3	ORD00002	CUST0016	2023-01-01	15:14	Nankhatai	7	171.99
4	ORD00002	CUST0016	2023-01-01	15:14	Paav	3	21.00

Categorias. Conferimos os valores de cada coluna de texto para procurar grafias diferentes da mesma categoria, espaços sobrando ou diferenças de maiúsculas e minúsculas.

In [7]:
```
colunas_texto = ["Customer_Gender", "Customer_Segment", "Loyalty_Member", "Day_of_Week",
"Weekend",
                 "Season", "Festival", "Weather", "Category", "Payment_Method",
"Promotion_Applied",
                 "Promotion_Type", "Storage_Type", "Perishability", "Store_ID"]

for coluna in colunas_texto:
    contagem = df_original[coluna].value_counts(dropna=False)
    print(f"{coluna} ({len(contagem)} valores): {contagem.to_dict()}")

textos = df_original.select_dtypes(include=["object", "string"])
espacos = {c: int((textos[c].dropna() != textos[c].dropna().str.strip()).sum()) for c in
textos}
variacoes_caixa = {c: int(textos[c].nunique() - textos[c].str.lower().nunique()) for c in
textos}
print("\nValores com espaço no início ou no fim:", sum(espacos.values()))
print("Categorias que só mudam por maiúscula/minúscula:", sum(variacoes_caixa.values()))

horario_sem_zero = ~df_original["Time"].str.fullmatch(r"\d{2}:\d{2}")
print(f"Horários sem zero à esquerda (ex.: '8:47'): {horario_sem_zero.sum()} "
      f"({horario_sem_zero.mean():.1%})")
```

```
Customer_Gender (3 valores): {'Female': 9084, 'Male': 8896, 'Other': 1341}
Customer_Segment (4 valores): {'Young Professional': 7245, 'Family': 5867, 'Student': 4932,
'Senior Citizen': 1277}
Loyalty_Member (2 valores): {'No': 11205, 'Yes': 8116}
Day_of_Week (7 valores): {'Sunday': 3613, 'Saturday': 3528, 'Monday': 2502, 'Friday': 2501,
'Wednesday': 2406, 'Tuesday': 2404, 'Thursday': 2367}
Weekend (2 valores): {'No': 12180, 'Yes': 7141}
Season (4 valores): {'Monsoon': 6466, 'Summer': 4831, 'Winter': 4777, 'Autumn': 3247}
Festival (10 valores): {nan: 16006, 'Ganesh Chaturthi': 967, 'Diwali': 438, 'Dussehra': 298,
'Eid': 290, 'Independence Day': 286, 'Christmas': 283, 'Republic Day': 278, 'Holi': 244, 'New
Year': 231}
Weather (6 valores): {'Clear': 5367, 'Humid': 4086, 'Rainy': 3392, 'Cold': 2454, 'Hot': 2365,
'Foggy': 1657}
Category (7 valores): {'Dairy & Spreads': 3620, 'Pastry & Sweet': 3577, 'Beverages': 3219,
'Indian Bakery': 3058, 'Bread & Daily': 2648, 'Cakes': 2189, 'Savoury Bakery': 1010}
Payment_Method (4 valores): {'UPI': 7040, 'Cash': 4670, 'Card': 4283, 'Wallet': 3328}
Promotion_Applied (2 valores): {'No': 13826, 'Yes': 5495}
Promotion_Type (6 valores): {nan: 13826, 'Festival': 2183, 'Weekend': 1607, 'Clearance': 853,
'Combo': 526, 'Flash Sale': 326}
Storage_Type (3 valores): {'Room Temperature': 6724, 'Refrigerated': 6596, 'Dry Storage': 6001}
Perishability (4 valores): {'Low': 6001, 'Very High': 5963, 'High': 5264, 'Medium': 2093}
Store_ID (3 valores): {'ST01': 6668, 'ST02': 6332, 'ST03': 6321}
Valores com espaço no início ou no fim: 0
Categorias que só mudam por maiúscula/minúscula: 0
Horários sem zero à esquerda (ex.: '8:47'): 5444 (28.2%)
```

Regras de consistência. Como várias colunas são calculadas a partir de outras, dá para testar se as contas fecham. Também conferimos se as informações que deveriam ser fixas (por pedido, por cliente, por produto e por dia) realmente não mudam. Cada regra abaixo conta quantas linhas a violam.

In [8]:
```
d = df_original
data = pd.to_datetime(d["Date"], format="%Y-%m-%d")
estacao_por_mes = {12: "Winter", 1: "Winter", 2: "Winter", 3: "Summer", 4: "Summer", 5:
"Summer",
                   6: "Monsoon", 7: "Monsoon", 8: "Monsoon", 9: "Monsoon", 10: "Autumn", 11:
"Autumn"}
```

```python
def varia_dentro_de(chave, colunas):
    return int((d.groupby(chave)[colunas].nunique() > 1).any(axis=1).sum())


regras = pd.DataFrame([
    ("Selling_Price = Unit_Price × (1 − desconto)",
     int(((d["Unit_Price"] * (1 - d["Discount_Percentage"] / 100)).round(2) -
d["Selling_Price"]).abs().gt(0.011).sum())),
    ("Total_Bill = Selling_Price × Quantity",
     int((d["Selling_Price"] * d["Quantity"] - d["Total_Bill"]).abs().gt(0.011).sum())),
    ("Profit = (Selling_Price − Cost_Price) × Quantity",
     int(((d["Selling_Price"] - d["Cost_Price"]) * d["Quantity"] -
d["Profit"]).abs().gt(0.011).sum())),
    ("Unsold_Units = Units_Produced − Units_Sold",
     int((d["Units_Produced"] - d["Units_Sold"] != d["Unsold_Units"]).sum())),
    ("Waste_Pct = Unsold_Units ÷ Units_Produced × 100",
     int((d["Unsold_Units"] / d["Units_Produced"] * 100 -
d["Waste_Pct"]).abs().gt(0.051).sum())),
    ("Year, Month e Day batem com Date",
     int(((data.dt.year != d["Year"]) | (data.dt.month != d["Month"]) | (data.dt.day !=
d["Day"])).sum())),
    ("Day_of_Week bate com Date", int((data.dt.day_name() != d["Day_of_Week"]).sum())),
    ("Weekend bate com Day_of_Week",
     int((d["Day_of_Week"].isin(["Saturday", "Sunday"]).map({True: "Yes", False: "No"}) !=
d["Weekend"]).sum())),
    ("Hour bate com Time", int((d["Time"].str.split(":").str[0].astype(int) !=
d["Hour"]).sum())),
    ("Season bate com o mês", int((d["Month"].map(estacao_por_mes) != d["Season"]).sum())),
    ("Dados do pedido fixos dentro do Order_ID",
     varia_dentro_de("Order_ID", ["Customer_ID", "Date", "Time", "Store_ID", "Payment_Method",
"Promotion_Type"])),
    ("Dados do cliente fixos por Customer_ID",
     varia_dentro_de("Customer_ID", ["Customer_Age", "Customer_Gender", "Customer_Segment",
"Loyalty_Member"])),
    ("Category, Storage_Type e Perishability fixos por produto",
     varia_dentro_de("Product", ["Category", "Storage_Type", "Perishability"])),
    ("Clima, estação e festival fixos por dia",
     varia_dentro_de("Date", ["Weather", "Temperature", "Season", "Festival"])),
    ("Promotion_Applied = No, mas Promotion_Type preenchido",
     int(((d["Promotion_Applied"] == "No") & d["Promotion_Type"].notna()).sum())),
    ("Promotion_Applied = Yes, mas Promotion_Type vazio",
     int(((d["Promotion_Applied"] == "Yes") & d["Promotion_Type"].isna()).sum())),
    ("Promotion_Applied = No, mas com desconto",
     int(((d["Promotion_Applied"] == "No") & (d["Discount_Percentage"] > 0)).sum())),
    ("Units_Sold menor que Quantity", int((d["Units_Sold"] < d["Quantity"]).sum())),
    ("Units_Sold diferente de Quantity", int((d["Units_Sold"] != d["Quantity"]).sum())),
    ("Stock_Available diferente de Unsold_Units", int((d["Stock_Available"] !=
d["Unsold_Units"]).sum())),
    ("Cost_Price maior que Selling_Price (lucro negativo)", int((d["Cost_Price"] >
d["Selling_Price"]).sum())),
], columns=["regra", "linhas que violam"])
regras["% da base"] = regras["linhas que violam"] / len(d) * 100

display(regras.style.format({"% da base": "{:.2f}"}).hide(axis="index"))
```

regra	linhas que violam	% da base
Selling_Price = Unit_Price × (1 − desconto)	0	0.00
Total_Bill = Selling_Price × Quantity	0	0.00
Profit = (Selling_Price − Cost_Price) × Quantity	0	0.00
Unsold_Units = Units_Produced − Units_Sold	0	0.00
Waste_Pct = Unsold_Units ÷ Units_Produced × 100	0	0.00
Year, Month e Day batem com Date	0	0.00
Day_of_Week bate com Date	0	0.00
Weekend bate com Day_of_Week	0	0.00
Hour bate com Time	0	0.00
Season bate com o mês	0	0.00
Dados do pedido fixos dentro do Order_ID	0	0.00
Dados do cliente fixos por Customer_ID	0	0.00
Category, Storage_Type e Perishability fixos por produto	0	0.00
Clima, estação e festival fixos por dia	0	0.00
Promotion_Applied = No, mas Promotion_Type preenchido	0	0.00
Promotion_Applied = Yes, mas Promotion_Type vazio	0	0.00
Promotion_Applied = No, mas com desconto	2089	10.81
Units_Sold menor que Quantity	0	0.00
Units_Sold diferente de Quantity	5161	26.71
Stock_Available diferente de Unsold_Units	1009	5.22
Cost_Price maior que Selling_Price (lucro negativo)	7	0.04

Algumas regras falharam e merecem um olhar mais de perto. Primeiro, os itens com desconto mas sem promoção registrada.

In [9]:
```
sem_promo_com_desconto = d[(d["Promotion_Applied"] == "No") & (d["Discount_Percentage"] > 0)]
print(f"Linhas: {len(sem_promo_com_desconto)} ({len(sem_promo_com_desconto) / len(d):.1%} da base)")
print("Desconto nesses casos:")
print(sem_promo_com_desconto["Discount_Percentage"].describe().round(2).to_string())

print("\nDesconto quando há promoção, por tipo:")
display(d[d["Promotion_Applied"] == "Yes"].groupby("Promotion_Type")["Discount_Percentage"]
        .describe()[["count", "min", "50%", "max"]].round(1))

marcador = (d["Promotion_Applied"] == "No") & (d["Discount_Percentage"] > 0)
print("Proporção desses casos por perecibilidade e por fidelidade (sem padrão claro):")
print(marcador.groupby(d["Perishability"]).mean().round(3).to_dict())
print(marcador.groupby(d["Loyalty_Member"]).mean().round(3).to_dict())
```

```
Linhas: 2089 (10.8% da base)
Desconto nesses casos:
count    2089.00
mean        5.23
std         2.21
min         1.00
25%         3.50
50%         5.30
75%         7.00
max        10.60

Desconto quando há promoção, por tipo:
```

Promotion_Type	count	min	50%	max
Clearance	853.0	15.0	23.3	30.0
Combo	526.0	5.0	10.6	17.5
Festival	2183.0	5.0	13.0	22.8
Flash Sale	326.0	10.0	18.0	27.1
Weekend	1607.0	3.0	8.2	14.7

```
Proporção desses casos por perecibilidade e por fidelidade (sem padrão claro):
{'High': 0.108, 'Low': 0.104, 'Medium': 0.128, 'Very High': 0.105}
{'No': 0.105, 'Yes': 0.112}
```

In [10]:
```
print("Itens com lucro negativo:")
display(d.loc[d["Profit"] < 0, ["Order_ID", "Product", "Quantity", "Discount_Percentage",
                                "Cost_Price", "Selling_Price", "Profit", "Promotion_Type"]])

print("Diferença entre Units_Sold e Quantity (Units_Sold - Quantity):")
print((d["Units_Sold"] - d["Quantity"]).value_counts().sort_index().head(8).to_dict())

produtos_vida_variavel = (d.groupby("Product")["Shelf_Life_Days"].nunique() > 1).sum()
print(f"\nProdutos com mais de um valor de Shelf_Life_Days: {produtos_vida_variavel} de
{d['Product'].nunique()}")
print("Alguns exemplos:")
display(d.groupby("Product")["Shelf_Life_Days"].agg(["min", "median", "max"])
        .loc[["Cold Coffee", "Bread", "Pastry", "Milk", "Cookies", "Honey"]])
```

```
Itens com lucro negativo:
```

	Order_ID	Product	Quantity	Discount_Percentage	Cost_Price	Selling_Price	Profit	Promotion_Type
1897	ORD00656	Milk	2	27.4	22.41	22.36	-0.10	Clearance
2531	ORD00872	Milk	1	28.6	22.46	21.91	-0.55	Clearance
2588	ORD00892	Milk	1	29.2	21.41	20.94	-0.47	Clearance
5520	ORD01895	Milk	1	27.7	22.34	22.20	-0.14	Clearance
6173	ORD02108	Paneer	1	27.4	58.78	58.60	-0.18	Clearance
7304	ORD02474	Milk	3	29.9	21.79	21.72	-0.21	Clearance
10420	ORD03510	Milk	1	28.4	22.62	22.38	-0.24	Clearance

```
Diferença entre Units_Sold e Quantity (Units_Sold - Quantity):
{0: 14160, 1: 1668, 2: 1480, 3: 609, 4: 436, 5: 257, 6: 203, 7: 126}

Produtos com mais de um valor de Shelf_Life_Days: 34 de 35
Alguns exemplos:
```

Product	min	median	max
Cold Coffee	1	1.0	1
Bread	3	4.0	5
Pastry	1	2.0	3
Milk	3	5.0	7
Cookies	30	78.0	120
Honey	182	471.0	729

Domínios e outliers. Verificamos se os valores numéricos estão dentro do que faz sentido (idade, temperatura, nota, hora, desconto e valores não negativos) e usamos a regra do intervalo interquartil (IQR) para apontar possíveis outliers. Para cada coluna mostramos as categorias de produto que concentram os casos marcados.

In [11]:
```
dominios = pd.DataFrame([
    ("Customer_Age entre 10 e 100", int((~d["Customer_Age"].between(10, 100)).sum())),
    ("Temperature entre -10 e 50 °C", int((~d["Temperature"].between(-10, 50)).sum())),
    ("Customer_Rating entre 1 e 5", int((~d["Customer_Rating"].between(1, 5)).sum())),
    ("Hour entre 0 e 23", int((~d["Hour"].between(0, 23)).sum())),
    ("Discount_Percentage entre 0 e 100", int((~d["Discount_Percentage"].between(0,
100)).sum())),
    ("Quantity, preços e Total_Bill maiores que zero",
     int((d[["Quantity", "Unit_Price", "Cost_Price", "Selling_Price", "Total_Bill"]] <=
0).any(axis=1).sum())),
    ("Units_Produced, Units_Sold e Shelf_Life_Days maiores que zero",
     int((d[["Units_Produced", "Units_Sold", "Shelf_Life_Days"]] <= 0).any(axis=1).sum())),
], columns=["regra de domínio", "linhas fora do domínio"])
display(dominios.style.hide(axis="index"))

linhas = []
```

```
for coluna in ["Quantity", "Unit_Price", "Total_Bill", "Profit", "Units_Produced",
               "Shelf_Life_Days", "Temperature", "Customer_Age"]:
    q1, q3 = d[coluna].quantile([0.25, 0.75])
    limite_inf, limite_sup = q1 - 1.5 * (q3 - q1), q3 + 1.5 * (q3 - q1)
    fora = d[(d[coluna] < limite_inf) | (d[coluna] > limite_sup)]
    linhas.append({
        "coluna": coluna,
        "limites IQR": f"{limite_inf:.1f} a {limite_sup:.1f}",
        "casos": len(fora),
        "% da base": len(fora) / len(d) * 100,
        "categorias mais frequentes": fora["Category"].value_counts().head(2).to_dict(),
    })
display(pd.DataFrame(linhas).style.format({"% da base": "{:.1f}"}).hide(axis="index"))
```

regra de domínio	linhas fora do domínio
Customer_Age entre 10 e 100	0
Temperature entre -10 e 50 °C	0
Customer_Rating entre 1 e 5	0
Hour entre 0 e 23	0
Discount_Percentage entre 0 e 100	0
Quantity, preços e Total_Bill maiores que zero	0
Units_Produced, Units_Sold e Shelf_Life_Days maiores que zero	0

coluna	limites IQR	casos	% da base	categorias mais frequentes
Quantity	-2.0 a 6.0	712	3.7	{'Indian Bakery': 384, 'Bread & Daily': 266}
Unit_Price	-37.4 a 145.1	2541	13.2	{'Cakes': 2099, 'Dairy & Spreads': 442}
Total_Bill	-89.5 a 288.2	2637	13.6	{'Cakes': 1801, 'Indian Bakery': 749}
Profit	-47.2 a 133.8	2280	11.8	{'Cakes': 1429, 'Indian Bakery': 787}
Units_Produced	-2.5 a 9.5	1298	6.7	{'Indian Bakery': 659, 'Bread & Daily': 273}
Shelf_Life_Days	-115.5 a 200.5	2340	12.1	{'Beverages': 1691, 'Dairy & Spreads': 649}
Temperature	3.2 a 51.2	0	0.0	{}
Customer_Age	-5.0 a 75.0	0	0.0	{}

In [12]:
```
variacao_preco = (d.groupby("Product")["Unit_Price"]
                  .agg(minimo="min", mediana="median", maximo="max")
                  .assign(variacao_pct=lambda t: ((t["maximo"] - t["minimo"]) / t["mediana"] *
100).round(1))
                  .sort_values("mediana"))
print("Preço de tabela por produto (cinco mais baratos e cinco mais caros):")
display(pd.concat([variacao_preco.head(5), variacao_preco.tail(5)]))
print(f"Maior variação de preço dentro de um mesmo produto:
{variacao_preco['variacao_pct'].max()}%")
```

```
Preço de tabela por produto (cinco mais baratos e cinco mais caros):
```

Product	minimo	mediana	maximo	variacao_pct
Paav	7.60	7.980	8.40	10.0
Puff	23.75	24.950	26.25	10.0
Tea Powder	28.50	29.950	31.50	10.0
Nankhatai	28.50	29.980	31.50	10.0
Biscuit	28.50	30.020	31.49	10.0
Honey	209.01	219.630	230.96	10.0
Pineapple Cake	380.51	399.470	419.73	9.8
Chocolate Cake	399.03	420.470	440.96	10.0
Butterscotch Cake	408.52	429.055	451.45	10.0
Black Forest Cake	427.54	449.480	472.42	10.0

```
Maior variação de preço dentro de um mesmo produto: 10.0%
```

Resposta do grupo:

- A base tem 19.321 linhas e 40 colunas, com 6.576 pedidos, 500 clientes, 35 produtos e 3 lojas. Cada linha é um produto dentro de um pedido, e os pedidos têm de 1 a 7 produtos.
- **Tipos e formatos:** `Date` e `Time` vieram como texto, e `Time` não tem zero à esquerda em 5.444 horários (28,2%, como `8:47`). `Loyalty_Member`, `Weekend` e `Promotion_Applied` são Yes/No em texto. Os identificadores já vieram como texto, e o restante dos números foi lido corretamente.
- **Ausentes:** só em duas colunas. `Festival` está vazio em 16.006 linhas (82,8%) e `Promotion_Type` em 13.826 (71,6%). Nos dois casos o vazio tem significado: é dia sem festival e item sem promoção. As 13.826 linhas sem tipo de promoção são exatamente as que têm `Promotion_Applied = No`.
- **Duplicatas:** não há linhas repetidas nem repetição de `Order_ID` + `Product`. As 12.745 repetições de `Order_ID` são esperadas, porque cada produto do pedido é uma linha. Se removêssemos "duplicatas" por `Order_ID`, perderíamos dois terços da base.
- **Categorias:** não encontramos grafias diferentes, espaços sobrando nem variações de maiúsculas e minúsculas.
- **Consistência:** todas as contas fecham (preço com desconto, total, lucro, sobra e percentual de sobra), as datas batem com ano, mês, dia, dia da semana, fim de semana e estação, e os dados de pedido, cliente, produto e dia não variam onde não deveriam. Os pontos que chamaram atenção foram estes:
  - **2.089 itens (10,8%) têm desconto, mas `Promotion_Applied = No`.** São descontos pequenos (de 1% a 10,6%), espalhados sem padrão por perecibilidade ou fidelidade. Como o preço de venda, o total e o lucro foram calculados com esse desconto, o desconto aconteceu; o que não bate é a marcação de promoção.
  - **7 itens têm lucro negativo.** Todos são leite ou paneer em promoção do tipo Clearance com desconto de 27% a 30%, então o desconto ficou maior que a margem. As contas estão certas.
  - **`Units_Sold` difere de `Quantity` em 5.161 linhas** e `Stock_Available` difere de `Unsold_Units` em 1.009. `Units_Sold` nunca é menor que `Quantity`, o que indica que essas colunas descrevem o lote de produção do produto, e não o pedido.
  - **`Shelf_Life_Days` varia dentro do mesmo produto** em 34 dos 35 produtos (o mel, por exemplo, vai de 182 a 729 dias). Lemos isso como a vida útil do lote, não como erro.
- **Domínios e outliers:** nenhum valor fora do domínio (idade de 16 a 73 anos, temperatura de 8,1 a 44,9 °C, notas de 1 a 5, vendas entre 7h e 21h). O IQR marca muitos "outliers" de preço, total e lucro, mas eles se concentram nos bolos, que custam cerca de 400 por unidade, enquanto o pão custa 40. Dentro de cada produto, o preço varia no máximo 10%. Quantidades altas (até 20 unidades) aparecem em itens baratos como Paav e Nankhatai.

4. Limpeza e preparação dos dados

Realize a limpeza necessária ao projeto. Verifique valores ausentes, duplicados e inconsistências e trate os problemas encontrados, quando necessário.

Antes de mexer nos dados, registramos as decisões e o motivo de cada uma. Seguimos a mesma lógica vista em aula: inspecionar, decidir, transformar e validar. Também registramos as decisões de **não** alterar os dados, que fazem parte do tratamento.

In [13]:
```
decisoes = pd.DataFrame([
    ("D1", "Repetição de Order_ID (12.745 linhas)",
     "Manter todas as linhas",
     "A chave do registro é Order_ID + Product, que não se repete. Remover por Order_ID
apagaria itens reais."),
    ("D2", "Festival vazio (16.006 linhas)",
     "Preencher com 'No Festival'",
     "O vazio significa dia sem festival. Um rótulo explícito evita que vire 'nan' em
agrupamentos."),
    ("D3", "Promotion_Type vazio (13.826 linhas)",
     "Preencher com 'No Promotion'",
     "Os vazios são exatamente os itens com Promotion_Applied = No."),
    ("D4", "Desconto sem promoção registrada (2.089 linhas)",
     "Manter o desconto e criar a coluna Discount_Without_Promotion",
     "Preço, total e lucro foram calculados com o desconto. Zerar o desconto deixaria essas
colunas erradas."),
    ("D5", "Date, Time e colunas Yes/No como texto",
     "Converter Date para data, padronizar Time em HH:MM e Yes/No para booleano",
     "Datas e horários precisam de tipo próprio para ordenar e montar o Timestamp."),
    ("D6", "Outliers pelo IQR em preço, total, lucro e quantidade",
     "Manter",
     "Refletem produtos caros (bolos) e compras maiores de itens baratos. As contas fecham e
os preços variam no máximo 10% por produto."),
    ("D7", "Lucro negativo (7 linhas)",
     "Manter",
     "É resultado real de Clearance com desconto maior que a margem. Interessa para a pergunta
P3."),
    ("D8", "Colunas de lote diferentes do pedido (Units_Sold, Stock_Available)",
     "Manter e documentar",
     "Descrevem o lote de produção, não o pedido. Não serão usadas para calcular o valor
vendido no pedido."),
    ("D9", "Shelf_Life_Days varia dentro do produto",
     "Manter",
     "Valores plausíveis para cada tipo de produto; lidos como vida útil do lote."),
    ("D10", "Domínios numéricos e categorias",
     "Nenhuma alteração",
     "Não há valores impossíveis nem categorias com grafias diferentes."),
], columns=["id", "problema", "decisão", "justificativa"])

display(decisoes.style.hide(axis="index").set_properties(**{"text-align": "left"}))
```

id	problema	decisão	justificativa
D1	Repetição de Order_ID (12.745 linhas)	Manter todas as linhas	A chave do registro é Order_ID + Product, que não se repete. Remover por Order_ID apagaria itens reais.
D2	Festival vazio (16.006 linhas)	Preencher com 'No Festival'	O vazio significa dia sem festival. Um rótulo explícito evita que vire 'nan' em agrupamentos.
D3	Promotion_Type vazio (13.826 linhas)	Preencher com 'No Promotion'	Os vazios são exatamente os itens com Promotion_Applied = No.
D4	Desconto sem promoção registrada (2.089 linhas)	Manter o desconto e criar a coluna Discount_Without_Promotion	Preço, total e lucro foram calculados com o desconto. Zerar o desconto deixaria essas colunas erradas.
D5	Date, Time e colunas Yes/No como texto	Converter Date para data, padronizar Time em HH:MM e Yes/No para booleano	Datas e horários precisam de tipo próprio para ordenar e montar o Timestamp.
D6	Outliers pelo IQR em preço, total, lucro e quantidade	Manter	Refletem produtos caros (bolos) e compras maiores de itens baratos. As contas fecham e os preços variam no máximo 10% por produto.
D7	Lucro negativo (7 linhas)	Manter	É resultado real de Clearance com desconto maior que a margem. Interessa para a pergunta P3.
D8	Colunas de lote diferentes do pedido (Units_Sold, Stock_Available)	Manter e documentar	Descrevem o lote de produção, não o pedido. Não serão usadas para calcular o valor vendido no pedido.
D9	Shelf_Life_Days varia dentro do produto	Manter	Valores plausíveis para cada tipo de produto; lidos como vida útil do lote.
D10	Domínios numéricos e categorias	Nenhuma alteração	Não há valores impossíveis nem categorias com grafias diferentes.

Todas as alterações são feitas em uma cópia, `df_limpo`. Cada tratamento mostra como estava antes e como ficou depois.

In [14]:
```
df_limpo = df_original.copy(deep=True)

# D1: duplicatas pela chave correta
antes = len(df_limpo)
df_limpo = df_limpo.drop_duplicates()
print("D1 - DUPLICATAS")
print(f"Linhas antes: {antes} | depois: {len(df_limpo)} | removidas: {antes - len(df_limpo)}")
print(f"Repetições de Order_ID + Product: {df_limpo.duplicated(subset=['Order_ID',
'Product']).sum()}")
```

```
D1 - DUPLICATAS
Linhas antes: 19321 | depois: 19321 | removidas: 0
Repetições de Order_ID + Product: 0
```

In [15]:
```
# D2 e D3: ausências que têm significado
print("D2 e D3 - AUSÊNCIAS")
print(f"Festival vazio antes: {df_limpo['Festival'].isna().sum()}")
print(f"Promotion_Type vazio antes: {df_limpo['Promotion_Type'].isna().sum()}")

df_limpo["Festival"] = df_limpo["Festival"].fillna("No Festival")
df_limpo["Promotion_Type"] = df_limpo["Promotion_Type"].fillna("No Promotion")

print(f"Festival vazio depois: {df_limpo['Festival'].isna().sum()}")
print(f"Promotion_Type vazio depois: {df_limpo['Promotion_Type'].isna().sum()}")
print("\nPromotion_Type x Promotion_Applied:")
display(pd.crosstab(df_limpo["Promotion_Type"], df_limpo["Promotion_Applied"]))
```

```
D2 e D3 - AUSÊNCIAS
Festival vazio antes: 16006
Promotion_Type vazio antes: 13826
Festival vazio depois: 0
Promotion_Type vazio depois: 0

Promotion_Type x Promotion_Applied:
```

Promotion_Applied	No	Yes
Promotion_Type		
Clearance	0	853
Combo	0	526
Festival	0	2183
Flash Sale	0	326
No Promotion	13826	0
Weekend	0	1607

In [16]:
```
# D4: desconto sem promoção registrada
df_limpo["Discount_Without_Promotion"] = (
    (df_limpo["Promotion_Applied"] == "No") & (df_limpo["Discount_Percentage"] > 0)
)

print("D4 - DESCONTO SEM PROMOÇÃO")
print(f"Itens marcados: {df_limpo['Discount_Without_Promotion'].sum()}")
print(f"Descontos alterados: 0 (o valor original foi mantido)")
print("\nResumo dos descontos por situação:")
situacao = np.select(
    [df_limpo["Promotion_Applied"] == "Yes", df_limpo["Discount_Without_Promotion"]],
    ["com promoção", "desconto sem promoção"], default="sem desconto")
display(df_limpo.groupby(situacao)["Discount_Percentage"].agg(["count", "min", "median",
"max"]))
```

```
D4 - DESCONTO SEM PROMOÇÃO
Itens marcados: 2089
Descontos alterados: 0 (o valor original foi mantido)

Resumo dos descontos por situação:
```

	count	min	median	max
com promoção	5495	3.0	11.8	30.0
desconto sem promoção	2089	1.0	5.3	10.6
sem desconto	11737	0.0	0.0	0.0

In [17]:
```
# D5: tipos de dados
tipos_antes = df_limpo[["Date", "Time", "Loyalty_Member", "Weekend",
"Promotion_Applied"]].dtypes.astype(str)
exemplo_time_antes = df_limpo["Time"].head(3).tolist()

df_limpo["Date"] = pd.to_datetime(df_limpo["Date"], format="%Y-%m-%d")
df_limpo["Time"] = pd.to_datetime(df_limpo["Time"], format="%H:%M").dt.strftime("%H:%M")

for coluna in ["Loyalty_Member", "Weekend", "Promotion_Applied"]:
    df_limpo[coluna] = df_limpo[coluna].map({"Yes": True, "No": False})

tipos_depois = df_limpo[["Date", "Time", "Loyalty_Member", "Weekend",
"Promotion_Applied"]].dtypes.astype(str)
print("D5 - TIPOS")
display(pd.DataFrame({"antes": tipos_antes, "depois": tipos_depois}))
print(f"Time antes: {exemplo_time_antes} | depois: {df_limpo['Time'].head(3).tolist()}")
print(f"Valores que não viraram booleano: "
      f"{df_limpo[['Loyalty_Member', 'Weekend', 'Promotion_Applied']].isna().sum().sum()}")
```

```
D5 - TIPOS
```

	antes	depois
Date	object	datetime64[ns]
Time	object	object
Loyalty_Member	object	bool
Weekend	object	bool
Promotion_Applied	object	bool

```
Time antes: ['8:47', '8:47', '15:14'] | depois: ['08:47', '08:47', '15:14']
Valores que não viraram booleano: 0
```

In [18]:
```
# D6 a D10: verificações que justificam manter os dados como estão
print("D6 a D10 - DADOS MANTIDOS")
print(f"Linhas com lucro negativo mantidas: {(df_limpo['Profit'] < 0).sum()} "
      f"(tipos: {df_limpo.loc[df_limpo['Profit'] < 0, 'Promotion_Type'].unique().tolist()})")
print(f"Linhas com Units_Sold < Quantity: {(df_limpo['Units_Sold'] <
df_limpo['Quantity']).sum()}")
print(f"Preço com variação acima de 10% dentro de um produto: "
      f"{(variacao_preco['variacao_pct'] > 10).sum()} produtos")
print(f"Linhas fora dos domínios definidos: {dominios['linhas fora do domínio'].sum()}")
```

```
D6 a D10 - DADOS MANTIDOS
Linhas com lucro negativo mantidas: 7 (tipos: ['Clearance'])
Linhas com Units_Sold < Quantity: 0
Preço com variação acima de 10% dentro de um produto: 0 produtos
Linhas fora dos domínios definidos: 0
```

Resposta do grupo:

Decisão	Problema	Tratamento	Efeito
D1	`Order_ID` repetido em 12.745 linhas	Nenhuma remoção. A chave é `Order_ID` + `Product`, que não se repete	0 linhas removidas
D2	`Festival` vazio em 16.006 linhas	Preenchido com `No Festival`	0 ausentes
D3	`Promotion_Type` vazio em 13.826 linhas	Preenchido com `No Promotion` (coincide com `Promotion_Applied = No`)	0 ausentes
D4	2.089 itens com desconto sem promoção registrada	Desconto mantido e nova coluna booleana `Discount_Without_Promotion`	+1 coluna, nenhum valor alterado
D5	`Date`, `Time` e colunas Yes/No como texto	`Date` em datetime, `Time` no formato `HH:MM` (5.444 horários ajustados), Yes/No em booleano	5 colunas com tipo ou formato corrigido
D6	Outliers pelo IQR	Mantidos: são produtos caros e compras maiores, com contas corretas	nenhuma alteração
D7	7 itens com lucro negativo	Mantidos: Clearance com desconto maior que a margem	nenhuma alteração
D8	`Units_Sold` e `Stock_Available` não batem com o pedido	Mantidos e documentados como informação do lote	nenhuma alteração
D9	`Shelf_Life_Days` varia dentro do produto	Mantido como vida útil do lote	nenhuma alteração
D10	Domínios e categorias	Verificados, sem problemas	nenhuma alteração

A decisão mais delicada foi a D4. Havia duas saídas: zerar o desconto para combinar com `Promotion_Applied = No` ou manter o desconto e marcar o caso. Zerar deixaria `Selling_Price`, `Total_Bill` e `Profit` errados em 2.089 linhas, porque essas colunas já foram calculadas com o desconto. Por isso mantivemos os valores e criamos a marcação, que permite separar esses casos na análise das promoções (P3).

Também evitamos dois erros que seriam fáceis de cometer nesta base: remover "duplicatas" por `Order_ID`, o que apagaria 66% dos itens, e converter os vazios de `Festival` com `astype(str)`, o que criaria uma categoria falsa chamada `nan`.

5. Transformação dos dados

Realize as transformações necessárias à análise, como criação de colunas, filtros, agrupamentos, conversões ou junções, quando aplicáveis.

In [19]:
```
df_preparado = df_limpo.copy()
```

```
df_preparado["Timestamp"] = pd.to_datetime(
    df_preparado["Date"].dt.strftime("%Y-%m-%d") + " " + df_preparado["Time"],
    format="%Y-%m-%d %H:%M",
)

# deixa o Timestamp ao lado de Date e Time e ordena as vendas no tempo
colunas = list(df_limpo.columns)
colunas.insert(colunas.index("Time") + 1, "Timestamp")
df_preparado = (df_preparado[colunas]
                .sort_values(["Timestamp", "Order_ID", "Product"])
                .reset_index(drop=True))

print(f"Timestamps vazios: {df_preparado['Timestamp'].isna().sum()}")
print(f"Primeira venda: {df_preparado['Timestamp'].min()} | última:
{df_preparado['Timestamp'].max()}")
print(f"Pedidos com mais de um horário: {(df_preparado.groupby('Order_ID')
['Timestamp'].nunique() > 1).sum()}")
df_preparado[["Order_ID", "Date", "Time", "Timestamp", "Product", "Quantity",
"Total_Bill"]].head()
```

```
Timestamps vazios: 0
Primeira venda: 2023-01-01 07:32:00 | última: 2024-12-31 19:00:00
Pedidos com mais de um horário: 0
```

Out[19]:

	Order_ID	Date	Time	Timestamp	Product	Quantity	Total_Bill
0	ORD00021	2023-01-01	07:32	2023-01-01 07:32:00	Bread	1	35.96
1	ORD00021	2023-01-01	07:32	2023-01-01 07:32:00	Muffin	3	123.93
2	ORD00021	2023-01-01	07:32	2023-01-01 07:32:00	Puff	7	159.46
3	ORD00014	2023-01-01	07:46	2023-01-01 07:46:00	Pastry	1	48.22
4	ORD00014	2023-01-01	07:46	2023-01-01 07:46:00	Vanilla Cupcake	1	31.65

Resposta do grupo:

Fizemos só transformações de preparo. Criamos a coluna `Timestamp`, que junta `Date` e `Time` no horário de cada venda; ela é a referência de tempo das perguntas por horário (P1) e será o tempo do evento no streaming do RA2. Depois ordenamos as vendas no tempo e posicionamos o `Timestamp` ao lado das colunas de origem. Todas as linhas geraram um horário válido, e cada pedido tem um único horário.

Não fizemos agregações, indicadores nem gráficos. Pelas orientações do RA1, isso fica para as próximas etapas.

6. Conjunto de dados resultante

Validamos a base preparada com regras explícitas e salvamos o arquivo que será o ponto de partida do RA2.

In [20]:

```
colunas_yes_no = ["Loyalty_Member", "Weekend", "Promotion_Applied"]

validacoes = pd.DataFrame([
    ("Nenhuma linha foi perdida", len(df_preparado) == len(df_original)),
    ("Order_ID + Product é único", not df_preparado.duplicated(subset=["Order_ID",
"Product"]).any()),
    ("Nenhum valor ausente", df_preparado.isna().sum().sum() == 0),
    ("Date e Timestamp são datas", pd.api.types.is_datetime64_any_dtype(df_preparado["Date"])
                                   and
pd.api.types.is_datetime64_any_dtype(df_preparado["Timestamp"])),
    ("Colunas Yes/No são booleanas", all(df_preparado[c].dtype == bool for c in
colunas_yes_no)),
    ("Time no formato HH:MM", df_preparado["Time"].str.fullmatch(r"\d{2}:\d{2}").all()),
    ("Sem 'nan' como texto em Festival e Promotion_Type",
```

```
     not df_preparado[["Festival", "Promotion_Type"]].isin(["nan", "Nan",
"NaN"]).any().any()),
    ("Promotion_Type = No Promotion quando não há promoção",
     ((df_preparado["Promotion_Type"] == "No Promotion") ==
~df_preparado["Promotion_Applied"]).all()),
    ("Descontos sem promoção marcados (2.089)",
df_preparado["Discount_Without_Promotion"].sum() == 2089),
    ("Total_Bill = Selling_Price × Quantity",
     (df_preparado["Selling_Price"] * df_preparado["Quantity"] -
df_preparado["Total_Bill"]).abs().max() <= 0.011),
    ("Valores financeiros iguais aos originais",
     np.isclose(df_preparado["Total_Bill"].sum(), df_original["Total_Bill"].sum())
     and np.isclose(df_preparado["Profit"].sum(), df_original["Profit"].sum())),
    ("Timestamp dentro do período da base",
     df_preparado["Timestamp"].between("2023-01-01", "2025-01-01").all()),
], columns=["regra verificada", "resultado"])
validacoes["status"] = validacoes["resultado"].map({True: "OK", False: "REVISAR"})
display(validacoes[["regra verificada", "status"]].style.hide(axis="index"))

falhas = validacoes.loc[~validacoes["resultado"], "regra verificada"].tolist()
assert not falhas, f"Validações que falharam: {falhas}"
print("Todas as regras foram atendidas.")
```

regra verificada	status
Nenhuma linha foi perdida	OK
Order_ID + Product é único	OK
Nenhum valor ausente	OK
Date e Timestamp são datas	OK
Colunas Yes/No são booleanas	OK
Time no formato HH:MM	OK
Sem 'nan' como texto em Festival e Promotion_Type	OK
Promotion_Type = No Promotion quando não há promoção	OK
Descontos sem promoção marcados (2.089)	OK
Total_Bill = Selling_Price × Quantity	OK
Valores financeiros iguais aos originais	OK
Timestamp dentro do período da base	OK

```
Todas as regras foram atendidas.
```

In [21]:
```
comparacao = pd.DataFrame([
    ("Linhas", len(df_original), len(df_preparado)),
    ("Colunas", df_original.shape[1], df_preparado.shape[1]),
    ("Valores ausentes", int(df_original.isna().sum().sum()),
int(df_preparado.isna().sum().sum())),
    ("Repetições de Order_ID + Product",
     int(df_original.duplicated(subset=["Order_ID", "Product"]).sum()),
     int(df_preparado.duplicated(subset=["Order_ID", "Product"]).sum())),
    ("Horários fora do formato HH:MM", int(horario_sem_zero.sum()),
     int((~df_preparado["Time"].str.fullmatch(r"\d{2}:\d{2}")).sum())),
    ("Colunas de data/hora com tipo de data", 0, 2),
    ("Colunas Yes/No como booleano", 0, 3),
    ("Descontos sem promoção identificados", 0,
int(df_preparado["Discount_Without_Promotion"].sum())),
], columns=["medida", "após ingestão", "após preparação"])
display(comparacao.style.hide(axis="index"))
```

medida	após ingestão	após preparação
Linhas	19321	19321
Colunas	40	42
Valores ausentes	29832	0
Repetições de Order_ID + Product	0	0
Horários fora do formato HH:MM	5444	0
Colunas de data/hora com tipo de data	0	2
Colunas Yes/No como booleano	0	3
Descontos sem promoção identificados	0	2089

In [22]:
```python
ARQUIVO_PREPARADO = Path("data/processed/bakery_sales_preparado.csv")
ARQUIVO_PREPARADO.parent.mkdir(parents=True, exist_ok=True)
df_preparado.to_csv(ARQUIVO_PREPARADO, index=False, encoding="utf-8")

# relê o arquivo para garantir que o que foi salvo é o que validamos
conferencia = pd.read_csv(ARQUIVO_PREPARADO, parse_dates=["Date", "Timestamp"])
assert conferencia.shape == df_preparado.shape
assert conferencia.isna().sum().sum() == 0
assert conferencia["Promotion_Applied"].dtype == bool

print(f"Arquivo salvo: {ARQUIVO_PREPARADO} ({ARQUIVO_PREPARADO.stat().st_size / 1_000_000:.1f} MB)")
print(f"{conferencia.shape[0]} linhas x {conferencia.shape[1]} colunas")
```

```
Arquivo salvo: data\processed\bakery_sales_preparado.csv (5.2 MB)
19321 linhas x 42 colunas
```

Resposta do grupo:

O resultado do RA1 é o arquivo `data/processed/bakery_sales_preparado.csv`, com 19.321 linhas e 42 colunas: as 40 originais mais `Timestamp` e `Discount_Without_Promotion`. Cada linha continua sendo um produto dentro de um pedido. As 12 regras de validação passaram: nenhuma linha foi perdida, a chave é única, não sobrou valor ausente, os tipos estão corretos e os valores financeiros são os mesmos da base original. Também relemos o arquivo salvo para confirmar que ele preserva o que foi validado.

7. Considerações do RA1

Registre brevemente:

- O que o grupo descobriu sobre os dados.
- Quais dificuldades ou limitações encontrou.
- Como os dados preparados poderão ser utilizados nas próximas etapas.

Resposta do grupo:

O que descobrimos. A base é bem organizada: as contas entre preço, desconto, total, lucro e sobra fecham em todas as linhas, e as informações de calendário, cliente, pedido e produto são consistentes. Os problemas reais foram poucos e pontuais: ausências com significado em `Festival` e `Promotion_Type`, tipos de data e Yes/No guardados como texto, e 2.089 itens com desconto sem a marcação de promoção. O ponto que mais exigiu atenção foi entender a granularidade: cada linha é um item do pedido, e não um pedido. Sem isso, a limpeza de duplicatas teria apagado dois terços dos dados.

Dificuldades e limitações.

- Os dados são sintéticos, então padrões encontrados não podem ser tratados como comportamento real de clientes.
- As colunas de produção e sobra (`Units_Produced`, `Units_Sold`, `Unsold_Units`, `Stock_Available`, `Waste_Pct`) descrevem o lote do produto e a base não explica como o lote é definido. Para a P4 vamos usar `Waste_Pct` e `Unsold_Units` como medida de sobra do lote, com esse cuidado.

- O clima é diário, o que limita a análise de clima por horário (P2).
- A moeda e a localização das lojas não são informadas.

Próximas etapas. A base preparada já tem o `Timestamp` de cada venda e os tipos corretos. No RA2 ela serve de referência para simular a chegada de novas vendas em streaming, e nas etapas seguintes será usada para agregações e indicadores por hora, loja, clima, festival, promoção e produto, respondendo às perguntas P1 a P4.

RA2 — Streaming Data Processing and Time Windows

Utilize os dados e o contexto do projeto para demonstrar o processamento em streaming e a aplicação de janelas temporais. Escolha um dos cenários:

- **Cenário A — Dados estáticos:** simule a chegada de novos registros utilizando dados artificiais coerentes com o contexto do projeto.
- **Cenário B — Dados em streaming:** utilize sua própria fonte de dados contínuos, sem necessidade de simulação.

8. Objetivo da análise em streaming

Responda brevemente:

- O que o grupo deseja monitorar ou analisar?
- Qual informação espera obter com o processamento em streaming?

Resposta do grupo:

Queremos acompanhar o movimento da rede durante o dia, enquanto as vendas acontecem: quanto foi faturado e quantos itens saíram em cada intervalo de uma hora, com o resultado atualizado a cada 30 minutos. Isso conversa com a pergunta P1 (variação das vendas ao longo do dia). Na prática, ajudaria a perceber o pico da manhã enquanto ele ainda está acontecendo e a decidir se é preciso repor produtos perecíveis, antes de esperar o relatório do fim do dia.

A informação esperada é uma tabela com o faturamento e a quantidade de itens de cada janela de tempo, que muda conforme novas vendas chegam.

9. Fonte de dados e processamento

- Informe o cenário escolhido: dados reais em streaming ou simulação.
- Explique brevemente como os novos registros chegam.
- Implemente o processamento em tempo real.
- Demonstre o recebimento de novos registros.

Resposta do grupo:

Cenário A (simulação). Nossa base é estática, então criamos vendas artificiais de um dia novo, segunda-feira, 06/01/2025, logo depois do fim da base. Para que sejam coerentes com o projeto, os produtos, as categorias e os preços vêm da base preparada no RA1: cada venda usa o preço de tabela mediano do produto.

Como os registros chegam. As vendas são publicadas em lotes, como se o sistema das lojas as enviasse ao longo da manhã. Cada lote é gravado como um arquivo JSON, com uma venda por linha, primeiro em uma pasta de preparo e depois movido para a pasta de entrada. Assim, o Spark só encontra arquivos completos.

Processamento. O Spark monitora a pasta de entrada com `readStream` e um schema explícito. Em cada venda que chega, aplicamos a mesma regra de qualidade do RA1 (descartar quantidade menor ou igual a zero) e calculamos `Selling_Price` e `Total_Bill` com as fórmulas que validamos na base. Primeiro iniciamos uma consulta que só mostra as vendas recebidas, para comprovar a chegada dos registros. A agregação por janela vem na seção 10.

In [23]:
```
import importlib.util
import json
import os
import platform
import subprocess
import sys
import tempfile
from uuid import uuid4

if importlib.util.find_spec("pyspark") is None:
    subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", "pyspark==4.0.3"])

# no Windows o Spark precisa do winutils.exe para ler a pasta e gravar o checkpoint (ver
README)
if platform.system() == "Windows" and "HADOOP_HOME" not in os.environ:
    os.environ["HADOOP_HOME"] = r"C:\hadoop"
    os.environ["PATH"] = r"C:\hadoop\bin;" + os.environ["PATH"]

from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.types import (DoubleType, IntegerType, StringType, StructField, StructType,
                               TimestampType)

spark = (
    SparkSession.builder
    .master("local[2]")
    .appName("Padaria - RA2")
    .config("spark.sql.session.timeZone", "America/Sao_Paulo")
    .config("spark.sql.shuffle.partitions", "2")
    .getOrCreate()
)
spark.conf.set("spark.sql.session.timeZone", "America/Sao_Paulo")
spark.sparkContext.setLogLevel("ERROR")
print("Spark", spark.version, "| fuso:", spark.conf.get("spark.sql.session.timeZone"))
```

```
Spark 4.0.3 | fuso: America/Sao_Paulo
```

O catálogo de produtos sai da base preparada no RA1. A função `gerar_venda` monta uma venda com o produto, a categoria e o preço de tabela vindos desse catálogo.

In [24]:
```
base_ra1 = pd.read_csv(ARQUIVO_PREPARADO, parse_dates=["Date", "Timestamp"])
catalogo = (base_ra1.groupby(["Product", "Category"])["Unit_Price"]
            .median().round(2).reset_index().set_index("Product"))
print(f"Produtos no catálogo: {len(catalogo)}")


def gerar_venda(id_venda, horario, loja, produto, quantidade, desconto=0.0):
    return {
        "Order_ID": id_venda,
        "Timestamp": f"2025-01-06 {horario}:00",
        "Store_ID": loja,
        "Product": produto,
        "Category": catalogo.loc[produto, "Category"],
        "Quantity": quantidade,
        "Unit_Price": float(catalogo.loc[produto, "Unit_Price"]),
        "Discount_Percentage": desconto,
    }


gerar_venda("V001", "07:10", "ST01", "Bread", 2)
```

```
Produtos no catálogo: 35
```

Out[24]:
```
{'Order_ID': 'V001',
 'Timestamp': '2025-01-06 07:10:00',
 'Store_ID': 'ST01',
 'Product': 'Bread',
 'Category': 'Bread & Daily',
 'Quantity': 2,
 'Unit_Price': 40.0,
 'Discount_Percentage': 0.0}
```

In [25]:
```
area_temporaria = tempfile.TemporaryDirectory(prefix="padaria_stream_",
ignore_cleanup_errors=True)
pasta_base = Path(area_temporaria.name)
pasta_entrada = pasta_base / "entrada"
pasta_preparo = pasta_base / "preparo"
pasta_entrada.mkdir()
pasta_preparo.mkdir()

schema_vendas = StructType([
    StructField("Order_ID", StringType()),
    StructField("Timestamp", TimestampType()),
    StructField("Store_ID", StringType()),
    StructField("Product", StringType()),
    StructField("Category", StringType()),
    StructField("Quantity", IntegerType()),
    StructField("Unit_Price", DoubleType()),
    StructField("Discount_Percentage", DoubleType()),
])

vendas_stream = (
    spark.readStream
    .schema(schema_vendas)
    .option("timestampFormat", "yyyy-MM-dd HH:mm:ss")
    .json(str(pasta_entrada))
)

vendas_processadas = (
    vendas_stream
    .filter(F.col("Quantity") > 0)
    .withColumn("Selling_Price",
                F.round(F.col("Unit_Price") * (1 - F.col("Discount_Percentage") / 100), 2))
    .withColumn("Total_Bill", F.round(F.col("Selling_Price") * F.col("Quantity"), 2))
)

print("É streaming?", vendas_processadas.isStreaming)
print("Arquivos na entrada:", list(pasta_entrada.iterdir()))
```

```
É streaming? True
Arquivos na entrada: []
```

In [26]:
```
nome_recebidas = "vendas_recebidas_" + uuid4().hex[:8]

consulta_recebidas = (
    vendas_processadas.writeStream
    .format("memory")
    .queryName(nome_recebidas)
    .outputMode("append")
    .option("checkpointLocation", str(pasta_base / "checkpoint_recebidas"))
    .start()
)

consultas = [consulta_recebidas]


def publicar_lote(vendas):
    nome_arquivo = f"lote_{uuid4().hex}.json"
    arquivo = pasta_preparo / nome_arquivo
    with arquivo.open("w", encoding="utf-8") as saida:
        for venda in vendas:
            saida.write(json.dumps(venda, ensure_ascii=False) + "\n")
    # só depois de fechado o arquivo vai para a pasta monitorada
    arquivo.replace(pasta_entrada / nome_arquivo)

    for consulta in consultas:
        consulta.processAllAvailable()
    print(f"Lote publicado com {len(vendas)} registro(s). Arquivos na entrada: "
          f"{len(list(pasta_entrada.glob('*.json')))}")


def mostrar_recebidas():
    (spark.table(nome_recebidas)
     .select("Order_ID", F.date_format("Timestamp", "HH:mm").alias("hora"), "Store_ID",
             "Product", "Quantity", "Total_Bill")
     .orderBy("Timestamp").show(truncate=False))


print("Consulta de recebimento ativa:", consulta_recebidas.isActive)
print("Vendas recebidas até agora:", spark.table(nome_recebidas).count())
```

```
Consulta de recebimento ativa: True
Vendas recebidas até agora: 0
```

Etapa 1: abertura das lojas. Chegam as quatro primeiras vendas, entre 7h10 e 7h55.

In [27]:
```
lote_1 = [
    gerar_venda("V001", "07:10", "ST01", "Bread", 2),
    gerar_venda("V002", "07:25", "ST02", "Milk", 1),
    gerar_venda("V003", "07:40", "ST03", "Paav", 6),
    gerar_venda("V004", "07:55", "ST01", "Puff", 3),
]
publicar_lote(lote_1)
mostrar_recebidas()
```

```
Lote publicado com 4 registro(s). Arquivos na entrada: 1
+--------+-----+--------+-------+--------+----------+
|Order_ID|hora |Store_ID|Product|Quantity|Total_Bill|
+--------+-----+--------+-------+--------+----------+
|V001    |07:10|ST01    |Bread  |2       |80.0      |
|V002    |07:25|ST02    |Milk   |1       |30.05     |
|V003    |07:40|ST03    |Paav   |6       |47.88     |
|V004    |07:55|ST01    |Puff   |3       |74.85     |
+--------+-----+--------+-------+--------+----------+
```

10. Aplicação de janelas temporais

- Utilize uma janela **Tumbling** (intervalos sem sobreposição) ou **Sliding** (intervalos que podem se sobrepor).
- Informe o tipo e a duração da janela e o deslocamento, quando aplicável.
- Implemente uma agregação coerente com a análise, como contagem, soma ou média.
- Explique brevemente o que será calculado em cada intervalo.

O foco é demonstrar a compreensão do conceito; não é necessário justificar tecnicamente a duração escolhida.

Resposta do grupo:

- **Tipo:** Sliding Window.
- **Duração:** 1 hora (`"1 hour"`).
- **Deslocamento:** 30 minutos (`"30 minutes"`). Uma janela nova começa a cada meia hora, então cada venda entra em duas janelas. A venda das 7h40, por exemplo, conta nas janelas 7h00–8h00 e 7h30–8h30.
- **Tempo usado:** o `Timestamp` da venda (tempo do evento), e não o horário em que o arquivo chegou. Por isso não precisamos esperar uma hora de verdade, e uma venda que chegar atrasada ainda cai na janela certa.
- **Agregação em cada janela:** soma de `Total_Bill` (faturamento), soma de `Quantity` (itens vendidos) e contagem de vendas.
- **Saída:** tabela em memória com `outputMode("complete")`, que mostra todas as janelas atualizadas depois de cada lote.

O início da janela entra e o fim não: uma venda às 8h00 pertence às janelas 7h30–8h30 e 8h00–9h00, e não à 7h00–8h00. Como as janelas se sobrepõem, somar o faturamento de todas as linhas não dá o total do dia.

In [28]:
```
faturamento_janelas = (
    vendas_processadas
    .groupBy(F.window("Timestamp", "1 hour", "30 minutes"))
    .agg(
        F.round(F.sum("Total_Bill"), 2).alias("faturamento"),
        F.sum("Quantity").alias("itens"),
        F.count("*").alias("vendas"),
    )
)

nome_janelas = "faturamento_janelas_" + uuid4().hex[:8]

consulta_janelas = (
    faturamento_janelas.writeStream
    .format("memory")
    .queryName(nome_janelas)
    .outputMode("complete")
    .option("checkpointLocation", str(pasta_base / "checkpoint_janelas"))
    .start()
)
consultas.append(consulta_janelas)


def mostrar_janelas():
    (spark.table(nome_janelas)
     .select(F.date_format("window.start", "HH:mm").alias("inicio"),
             F.date_format("window.end", "HH:mm").alias("fim"),
```

```
            "faturamento", "itens", "vendas")
     .orderBy("inicio").show(truncate=False))


# a consulta nova lê o lote 1, que já estava na pasta, e continua esperando arquivos
consulta_janelas.processAllAvailable()
print("Consulta das janelas ativa:", consulta_janelas.isActive)
mostrar_janelas()
```

```
Consulta das janelas ativa: True
+------+-----+-----------+-----+------+
|inicio|fim  |faturamento|itens|vendas|
+------+-----+-----------+-----+------+
|06:30 |07:30|110.05     |3    |2     |
|07:00 |08:00|232.78     |12   |4     |
|07:30 |08:30|122.73     |9    |2     |
+------+-----+-----------+-----+------+
```

11. Resultados e interpretação

Apresente os resultados do processamento e demonstre como a análise se comporta com a chegada de novos registros.

Etapa 2: pico da manhã. Chegam quatro vendas entre 8h05 e 8h50, entre elas um bolo de chocolate e um pastry com 10% de desconto (combo).

In [29]:
```
lote_2 = [
    gerar_venda("V005", "08:05", "ST02", "Chocolate Cake", 1),
    gerar_venda("V006", "08:20", "ST03", "Tea Powder", 1),
    gerar_venda("V007", "08:35", "ST01", "Pastry", 2, desconto=10.0),
    gerar_venda("V008", "08:50", "ST02", "Cold Coffee", 2),
]
publicar_lote(lote_2)
mostrar_janelas()
```

```
Lote publicado com 4 registro(s). Arquivos na entrada: 2
+------+-----+-----------+-----+------+
|inicio|fim  |faturamento|itens|vendas|
+------+-----+-----------+-----+------+
|06:30 |07:30|110.05     |3    |2     |
|07:00 |08:00|232.78     |12   |4     |
|07:30 |08:30|573.15     |11   |4     |
|08:00 |09:00|698.08     |6    |4     |
|08:30 |09:30|247.66     |4    |2     |
+------+-----+-----------+-----+------+
```

Etapa 3: final da manhã, com um registro inválido. Chegam três vendas. A V011 veio com quantidade zero (um erro de registro no caixa) e deve ser descartada pelo filtro.

In [30]:
```
lote_3 = [
    gerar_venda("V009", "09:10", "ST03", "Black Forest Cake", 1),
    gerar_venda("V010", "09:30", "ST01", "Donut", 4),
    gerar_venda("V011", "09:45", "ST02", "Bread", 0),
]
publicar_lote(lote_3)
mostrar_janelas()
print("Vendas recebidas até agora (a V011 não aparece):")
mostrar_recebidas()
```

```
Lote publicado com 3 registro(s). Arquivos na entrada: 3
+------+-----+-----------+-----+------+
|inicio|fim  |faturamento|itens|vendas|
+------+-----+-----------+-----+------+
|06:30 |07:30|110.05     |3    |2     |
|07:00 |08:00|232.78     |12   |4     |
|07:30 |08:30|573.15     |11   |4     |
|08:00 |09:00|698.08     |6    |4     |
|08:30 |09:30|697.14     |5    |3     |
|09:00 |10:00|609.16     |5    |2     |
|09:30 |10:30|159.68     |4    |1     |
+------+-----+-----------+-----+------+

Vendas recebidas até agora (a V011 não aparece):
+--------+-----+--------+-----------------+--------+----------+
|Order_ID|hora |Store_ID|Product          |Quantity|Total_Bill|
+--------+-----+--------+-----------------+--------+----------+
|V001    |07:10|ST01    |Bread            |2       |80.0      |
|V002    |07:25|ST02    |Milk             |1       |30.05     |
|V003    |07:40|ST03    |Paav             |6       |47.88     |
|V004    |07:55|ST01    |Puff             |3       |74.85     |
|V005    |08:05|ST02    |Chocolate Cake   |1       |420.47    |
|V006    |08:20|ST03    |Tea Powder       |1       |29.95     |
|V007    |08:35|ST01    |Pastry           |2       |107.96    |
|V008    |08:50|ST02    |Cold Coffee      |2       |139.7     |
|V009    |09:10|ST03    |Black Forest Cake|1       |449.48    |
|V010    |09:30|ST01    |Donut            |4       |159.68    |
+--------+-----+--------+-----------------+--------+----------+
```

Etapa 4: venda atrasada. Chega uma venda das 10h05 e, junto com ela, uma venda das 8h40 que estava presa no caixa da loja ST03 (por exemplo, por falta de internet). Como as janelas usam o horário da venda, a venda atrasada atualiza janelas que já tinham resultado.

In [31]:
```python
lote_4 = [
    gerar_venda("V012", "08:40", "ST03", "Butterscotch Cake", 1),
    gerar_venda("V013", "10:05", "ST01", "Milk", 2),
]
publicar_lote(lote_4)
mostrar_janelas()
```

```
Lote publicado com 2 registro(s). Arquivos na entrada: 4
+------+-----+-----------+-----+------+
|inicio|fim  |faturamento|itens|vendas|
+------+-----+-----------+-----+------+
|06:30 |07:30|110.05     |3    |2     |
|07:00 |08:00|232.78     |12   |4     |
|07:30 |08:30|573.15     |11   |4     |
|08:00 |09:00|1127.14    |7    |5     |
|08:30 |09:30|1126.2     |6    |4     |
|09:00 |10:00|609.16     |5    |2     |
|09:30 |10:30|219.78     |6    |2     |
|10:00 |11:00|60.1       |2    |1     |
+------+-----+-----------+-----+------+
```

Para conferir o resultado, refazemos o mesmo cálculo em modo batch (`spark.read`), lendo de uma vez todos os arquivos que chegaram. Se o streaming estiver correto, as duas tabelas precisam ser iguais.

In [32]:
```python
batch = (
    spark.read.schema(schema_vendas)
    .option("timestampFormat", "yyyy-MM-dd HH:mm:ss")
    .json(str(pasta_entrada))
    .filter(F.col("Quantity") > 0)
```

```
    .withColumn("Selling_Price",
                F.round(F.col("Unit_Price") * (1 - F.col("Discount_Percentage") / 100), 2))
    .withColumn("Total_Bill", F.round(F.col("Selling_Price") * F.col("Quantity"), 2))
    .groupBy(F.window("Timestamp", "1 hour", "30 minutes"))
    .agg(F.round(F.sum("Total_Bill"), 2).alias("faturamento"),
         F.sum("Quantity").alias("itens"),
         F.count("*").alias("vendas"))
)

resultado_stream =
spark.table(nome_janelas).toPandas().sort_values("window").reset_index(drop=True)
resultado_batch = batch.toPandas().sort_values("window").reset_index(drop=True)
print("Streaming e batch iguais?", resultado_stream.equals(resultado_batch))

total_dia = spark.table(nome_recebidas).agg(F.round(F.sum("Total_Bill"),
2).alias("t")).first()["t"]
print(f"Faturamento real da manhã (soma das vendas): {total_dia}")
print(f"Soma das janelas (conta cada venda duas vezes):
{resultado_stream['faturamento'].sum():.2f}")
```

```
Streaming e batch iguais? True
Faturamento real da manhã (soma das vendas): 2029.18
Soma das janelas (conta cada venda duas vezes): 4058.36
```

In [33]:

```
print("Micro-batches processados pela consulta das janelas:")
for progresso in consulta_janelas.recentProgress:
    if progresso["numInputRows"] > 0:
        print(f"  batch {progresso['batchId']}: {progresso['numInputRows']} registro(s)
processados")

for consulta in consultas:
    consulta.stop()
spark.catalog.dropTempView(nome_recebidas)
spark.catalog.dropTempView(nome_janelas)
area_temporaria.cleanup()
print("Consultas ativas:", [c.isActive for c in consultas])
```

```
Micro-batches processados pela consulta das janelas:
  batch 0: 4 registro(s) processados
  batch 1: 4 registro(s) processados
  batch 2: 2 registro(s) processados
  batch 3: 2 registro(s) processados
Consultas ativas: [False, False]
```

Resposta do grupo:

Chegaram 13 vendas em quatro lotes. Doze passaram pelo filtro; a V011, com quantidade zero, foi descartada, como faríamos no RA1. Cada lote virou um micro-batch. No terceiro aparecem 2 registros em vez de 3 porque a V011 já sai no filtro, durante a leitura.

Como a tabela mudou a cada lote:

- **Etapa 1:** surgem três janelas. A de 7h00–8h00 reúne as quatro vendas da abertura (faturamento de 232,78 e 12 itens). As janelas 6h30–7h30 e 7h30–8h30 dividem essas mesmas vendas, porque cada venda entra em duas janelas.
- **Etapa 2:** a janela 7h30–8h30 sobe de 122,73 para 573,15 com o bolo de chocolate das 8h05, e aparecem as janelas 8h00–9h00 (698,08) e 8h30–9h30 (247,66). As janelas anteriores ficam iguais, porque nenhuma venda nova caiu nelas.
- **Etapa 3:** as vendas das 9h10 e 9h30 levam a janela 8h30–9h30 a 697,14 e abrem 9h00–10h00 e 9h30–10h30. A venda inválida não muda nada.
- **Etapa 4:** a venda atrasada das 8h40 (bolo butterscotch, 429,06) chegou depois das vendas das 9h, mas foi somada às janelas do horário em que aconteceu: 8h00–9h00

passou de 698,08 para 1.127,14 e 8h30–9h30 de 697,14 para 1.126,20. A venda das 10h05 atualizou 9h30–10h30 e abriu 10h00–11h00.

O que os números mostram. O maior faturamento ficou na janela 8h00–9h00 (1.127,14), puxado por dois bolos. Já a maior quantidade de itens ficou na 7h00–8h00 (12 itens, quase todos baratos, como pão e paav). Olhar só a quantidade ou só o faturamento apontaria picos diferentes, e para a produção os dois importam.

Conferência. A tabela final do streaming é igual à do mesmo cálculo feito em batch, então nenhuma venda foi perdida ou contada a mais. O faturamento real da manhã foi 2.029,18, e a soma das janelas dá 4.058,36, exatamente o dobro, porque com duração de 1 hora e deslocamento de 30 minutos cada venda pertence a duas janelas.

12. Considerações do RA2

Responda brevemente:

- O que foi possível observar com o processamento em streaming?
- Qual foi a contribuição das janelas temporais para a análise?
- Essa abordagem faria sentido em uma aplicação real do projeto? Por quê?

Na última resposta, o grupo pode reconhecer que o streaming foi utilizado apenas como exercício didático, se esse for o caso.

Resposta do grupo:

O que observamos. O Spark trata as vendas que chegam como linhas novas de uma tabela que só cresce. Cada arquivo publicado virou um micro-batch, e a tabela de resultado foi atualizada sem precisarmos rodar nada de novo. Para isso, o Spark guarda entre um lote e outro a soma de cada janela (processamento com estado) e registra o progresso no checkpoint. As transformações foram as mesmas que usaríamos em batch (`filter` , `withColumn` , `groupBy`), e o resultado final bateu com o cálculo em batch.

Contribuição das janelas. As janelas transformam um fluxo contínuo em intervalos que podem ser comparados entre si. Com a sliding window, a leitura é atualizada a cada 30 minutos, mas sempre olhando para a última hora inteira, o que evita saltos bruscos de um intervalo para o outro. Usar o horário da venda, e não o horário de chegada, fez a venda atrasada cair na janela certa. O cuidado é não somar janelas que se sobrepõem para obter um total.

Faria sentido em uma aplicação real? Em parte. Numa rede de padarias com movimento grande, acompanhar o faturamento e os itens vendidos por hora durante o dia ajudaria a decidir reposição de produtos perecíveis e escala de funcionários. Mas a nossa base tem, em média, 9 pedidos por dia somando as três lojas. Com esse volume, um relatório em batch a cada hora ou no fim do dia resolveria o problema com muito menos complexidade. Por isso, neste projeto o streaming foi principalmente um exercício didático. Num uso real, as vendas viriam do sistema de caixa (por exemplo, por uma fila

de mensagens), usaríamos watermark para descartar o estado de janelas antigas e gravaríamos o resultado em um banco de dados em vez da memória.